Juha Laurila / Fullstack blog
===

Introduction To Web Development

Assignment 1:
Build a simple "Hello world!" webserver with node.js and express.js

- Dockerizing a Node.js web app

As a getting started task, I created files for dockerizing a simple node.js web application. Following instructions, step by step, I managed to implement very simple app in Docker container, what had be called with curl command. That was a good beginning to learn more of modern web design and Docker.

- Simple Web Server Assignment

Second step was creating a web application running in a container, which would be possible to call by web browser. Again, according to instructions, I created three necessary files for working application. In server.js file, there are defined a web app which uses the Express.js framework, the port has determined to be 3000 and the response to request of user is simply the text: "Hello h2c students!". Next I created a package.json file to meet some dependencies for the node.js applications.
At last, I created a new Dockerfile, which is a text document that contains all the commands a user could call on the command line to build a Docker image. When the new build image will be run, a new container will be launch. After that, when I wrote in a browser http://localhost:3000, I got the response “Hello H2C Students!”, so the task succeeded.
https://gitlab.dclabra.fi/JuhaLau/juhalaurila-fullstack

Assignment 2: Telnet and IRC

In followng assignment I had to join the IRC network over the Telnet protocol.
- Dockerizing

In this assigment, the first step was to build up a Docker container to start a Linux environment in a container. Althought the building of Linux container was not necessary completing this task in principle, it was very useful to learn, how to easily and quickly set up running environments on Docker.
All the necessities had be built in bash commands - separate files to complete this task had not be needed.
At first, I named a container and run Linux 18.04 Docker image with a command: docker run -it --name telnet-test ubuntu:18.04 bash.
- Installing and testing Telnet connection

After that, I installed Telnet with commands:

apt update
apt install telnet

Now I have a running container with Linux and Telnet. Finally, I wrote following commannd in Git bash: telnet irc.quakenet.org 6667; so I noticed the Telnet connection established.

https://gitlab.dclabra.fi/juhalau/juhalaurila-a2_telnet-and-irc

Juha Laurila / Fullstack blog
===

Introduction To Web Development

Assignment 3: Simple chat server over TCP

In this assigment, the first task were to be implement a simple echo server using Telnet. To complete that, I created two files, echoserver.js and Dockerfile. In the server.js file, there have been determined the handling of TCP connection, created a server to listening for requests and echo message to the user. Two last rows have been dedicated to log messages of console.

Another file, Dockerfile is a text file that consists of instructions to build Docker images. To build a container, I launched first Docker program and then I pasted "docker build -t test/tcp-echo-server ." and then "docker run --name tcp-echo-server -p 3000:3000 test/tcp-echo-server". After that I started another terminal and connected to new container with command "docker exec -it tcp-echo-server sh". Then I wrote "apk update" to get the list of available packages in Alpine Linux repositories. Then I wrote "apk add busybox-extras" to add Telnet available. Finally I wrote "Telnet localhost 3000" and the Telnet connection opened.
https://gitlab.dclabra.fi/juhalau/fullstack_echo_server

The second phase in the assigment was to test the application with multiple Telnet-connections. I have to create new chatserver.js file, which would enable multiple connections at same time. If previous echoserver.js file is such as a "sceleton file" as an server.file, this new one contains much more properties in addition.

https://gitlab.dclabra.fi/juhalau/juha-laurila_simple-chat-server-over-tcp

Assignment 4: Going Full Stack (finally!)

Eventually I was ready to move to do my first fullstack assignment with three main entities: client side implementation (UI), server side implementation and database (MongoDB). The objective of the assignment was to learn basics of implementing a fullstack application in a containers with node.js and database and finally build up the whole application by one command.

- Client files

At first, according to instructions, I began to create client side implentation of the project. There were two separate files required to the client implementation: index.html for login page + loading presidents and client.js for client application. Both of them located to folder app/client/. In the index.html there hs been determined the basic structure of UI, menu structure made with jquery library and other locations for JavaScript functionalities implemented in client.js file (for example <button id="load">Load presidents</button>) and fields for incoming data from database.

Another file, client.js is for loading an application, including status messages to console, determining and binding with Ajax call functionalities for the button of "Load presidents" and implementation accurately layout, presentation and elements for incoming data from database.

- Server

Secondly I began to work with server.js file, which contains the Web server and the REST API for handling the HTTP requests from the client and also the integration with the database, which in application like this is absolutely necessary. In the server.js file, there are also determined a listening port for requests of users and codes for status messages of a console. The server.js file has also placed to app folder.

- Database

Thirdly, to be done all the most important part of full stack implementation ready to dockerizing, I created necessary files for database.

To define a Mongoose schema (i.e. a structure) for using MongoDB - the database used in this project, I created the president.js file and placed it into model subfolder in app folder, that is the folder dedicated for database files. Mongoose is a JavaScript framework commonly used in Node.js applications with the MongoDB database. President.js creates the model and validating for presidents and their data fields used in database, such as name, nickname, image and so on.

The file to implement actual database is another file, app/database.js. In the database.js has been created routines to connect to Mongodb database, necessary structures and console messages - also error messages in possible fail situations and other functionalities and interfaces to necessary files to implement the database.

Finally I needed to define the dependencies - such as links to cors, express and mongoose libraries - required by the full stack application in package.json file that is placed in app folder.

The database was populated with a file Presidents.json. This file contains only the exact information for each president, which is included to a database. Writing the data of this file to the database is done later by command curl using the REST API our application provides. But before that I have to create containers for application.

- Dockerizing the server application

Following step in the project was dockerizing the server application. This is made with Dockerfile file placed in app folder.
I build it with git bash command: "docker build -t kamk/fullstack-example"
and run it with command: "docker run -it kamk/fullstack-example". In Docker desktop program, I noticed also that just created app container was running.

- Creating utilities container

I had to create also container to populate data to database. So I created another Dockerfile file containing required things to implement these functionalities and I saved it to utilities folder, alongside the application folder.

The last step was to create Docker Compose script with docker-compose.yml file to start both containers on a single command. I done that with command: "docker-compose up --build".
So, I have now both the containers running.

Because previous command reserved the current terminal, I had to open another terminal for next command to connect to utilities container.
I operated it with command "docker exec -it presidents-utilities bash".

Then it was time to populate the database with the information of presidents. The last command was:
"curl -H "Accept: application/json" -H "Content-type: application/json" -X POST -d '@presidents.json' presidents-server:8080/api/v1/presidents".

Now I had done all necessary things to implementation of my first full stack application. When I wrote to my web browser http://localhost:8080, and pressed the "Load presidents" button, the wonder happened and I saw the President page without problem, eventually!

https://gitlab.dclabra.fi/juhalau/fullstack_presidents